Manual Técnico del Sistema TuEvento

Equipo de Desarrollo TuEvento

3 de octubre de 2025

1 Introducción

1.1 Propósito del Documento

Este manual técnico proporciona una documentación completa del sistema TuEvento, una plataforma integral para la gestión y organización de eventos. El documento está dirigido a desarrolladores, administradores de sistemas, y personal técnico involucrado en el mantenimiento, despliegue y evolución del sistema.

1.2 Alcance del Sistema

TuEvento es una plataforma web y móvil que permite:

- Gestión completa de eventos por parte de organizadores
- Descubrimiento y reserva de eventos por parte de usuarios finales
- Sistema administrativo para moderación y control
- Diseño interactivo de layouts de eventos
- Sistema de calificaciones y reseñas

2 Visión General del Sistema

2.1 Arquitectura General

TuEvento sigue una arquitectura de microservicios distribuida con separación clara entre frontend y backend:

- **Frontend Web**: Aplicación React SPA responsiva
- **Frontend Móvil**: Aplicación React Native con Expo
- **Backend**: API RESTful desarrollada en Spring Boot
- **Base de Datos**: PostgreSQL relacional
- **Almacenamiento**: AWS S3 para archivos multimedia
- **Email Service**: SMTP para notificaciones

2.2 Tecnologías Principales

2.2.1 Backend

- **Framework**: Spring Boot 3.3.4
- **Lenguaje**: Java 17
- **Base de Datos**: PostgreSQL
- **Seguridad**: Spring Security + JWT
- **Documentación API**: OpenAPI/Swagger
- **Almacenamiento**: AWS S3 SDK
- **Email**: JavaMail

2.2.2 Frontend Web

- **Framework**: React 19.1.1
- **Lenguaje**: JavaScript ES6+
- **Enrutamiento**: React Router DOM 7.8.1
- **Estilos**: Tailwind CSS 4.1.12
- **Animaciones**: Framer Motion 12.23.16
- **Build Tool**: Vite

2.2.3 Frontend Móvil

- **Framework**: React Native con Expo
- **Lenguaje**: JavaScript/TypeScript
- **Navegación**: React Navigation

2.3 Historias de Usuario Principales

Basado en las historias de usuario definidas, el sistema soporta los siguientes flujos principales:

2.3.1 Autenticación y Registro

1. Creación de cuentas de usuario
2. Verificación de email
3. Inicio de sesión persistente
4. Recuperación de contraseña

2.3.2 Gestión de Perfil

1. Visualización y edición de información personal
2. Activación/desactivación de cuenta
3. Eliminación permanente de cuenta

2.3.3 Descubrimiento de Eventos

1. Listado de eventos disponibles
2. Búsqueda por nombre
3. Filtrado por categoría, ubicación y fecha
4. Visualización detallada de eventos

2.3.4 Gestión de Eventos (Organizadores)

1. Creación de nuevos eventos
2. Edición de eventos existentes
3. Cancelación de eventos
4. Panel de control de eventos

2.3.5 Peticiones de Organizadores

1. Solicitud de rol de organizador
2. Revisión administrativa de solicitudes
3. Aprobación/rechazamiento de solicitudes

2.3.6 Diseño de Asientos y Layouts

1. Diseño interactivo de layouts
2. Gestión de secciones y asientos
3. Configuración de capacidades

2.3.7 Experiencia de Usuario

1. Interfaces responsivas web y móvil
2. Navegación intuitiva
3. Diseño moderno y accesible

3 Arquitectura del Sistema

3.1 Patrones de Diseño Implementados

3.1.1 Backend

- **MVC Pattern**: Patrón Modelo-Vista-Controlador
- **Repository Pattern**: Abstracción del acceso a datos
- **Service Layer**: Lógica de negocio centralizada
- **Dependency Injection**: Inyección de dependencias con Spring
- **DTO Pattern**: Transferencia de datos entre capas
- **Exception Handling**: Manejo centralizado de excepciones

3.1.2 Frontend

- **Component Pattern**: Componentes reutilizables
- **Hooks Pattern**: Lógica reutilizable con custom hooks
- **Context API**: Estado global compartido
- **Container/Presentational**: Separación de lógica y presentación

4 Backend - Arquitectura Detallada

4.1 Estructura del Proyecto

La estructura del proyecto backend sigue las convenciones de Spring Boot:
La estructura del proyecto backend sigue las convenciones estándar de Spring Boot:

- `TuEventoBackendApplication.java`: Clase principal de Spring Boot
- `config/`: Configuraciones de seguridad, CORS, Swagger, almacenamiento
- `controller/`: Controladores REST para endpoints de la API
- `dto/`: Objetos de transferencia de datos
- `jwt/`: Utilidades para autenticación JWT
- `model/`: Entidades JPA del modelo de datos
- `repository/`: Interfaces de repositorio JPA
- `service/`: Lógica de negocio y servicios
- `data/`: Datos iniciales para desarrollo

4.2 Configuraciones Principales

4.2.1 application.properties

```
# Base de datos
spring.datasource.url=jdbc:postgresql://localhost:5432/tuevento
spring.datasource.username=postgres
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# JWT
jwt.secret=mySecretKey
jwt.expiration=86400000

# AWS S3
aws.s3.bucket-name=tuevento-bucket
aws.s3.region=us-east-1
aws.access.key=your-access-key
aws.secret.key=your-secret-key

# Email
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your-email@gmail.com
spring.mail.password=your-app-password
```

Listing 1: Configuración Principal

4.2.2 SecurityConfig.java

Configuración de seguridad con Spring Security:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
    Exception {
        http.csrf().disable()
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/swagger-ui/**").permitAll()
                .requestMatchers("/v3/api-docs/**").permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(sess -> sess.sessionCreationPolicy(
    SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtAuthenticationFilter(),
    UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Listing 2: Configuración de Seguridad

4.3 Modelo de Datos

4.3.1 Entidad User

```java
@Entity(name = "app_user")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "userID")
    private int userID;

    @Column(name = "fullName", length = 70, nullable = false)
    private String fullName;

    @Column(name = "telephone", length = 11)
    private String telephone;

    @Column(name = "status", nullable = false)
    private boolean status;

    @Column(name = "activated", nullable = false)
    private boolean activated;

    @Enumerated(EnumType.STRING)
    @Column(name = "role")
    private Role role;

    @Column(name = "birthDate")
    private Date birthDate;

    @ManyToOne
    @JoinColumn(name = "addressID")
    private Address address;

    @Column(name = "organicer", nullable = false)
    private boolean organicer;

    // Getters y setters...
}
```

Listing 3: Entidad User

4.3.2 Entidad Event

```java
@Entity(name="event")
public class Event {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "eventID")
    private int id;

    @ManyToOne
    @JoinColumn(name="userID", nullable=false)
    private User userID;

    @ManyToOne
    @JoinColumn(name="locationID", nullable=false)
```

```
    private Location locationID;

    @OneToMany(mappedBy = "event", cascade = CascadeType.REMOVE)
    private List<EventImg> images;

    @OneToMany(mappedBy = "event", cascade = CascadeType.REMOVE)
    private List<CategoryEvent> categories;

    @Column(name="eventName", length = 100, nullable = false)
    private String eventName;

    @Column(name="description", length = 500, nullable = false)
    private String description;

    @Column(name="startDate", nullable = false)
    private LocalDate startDate;

    @Column(name="finishDate", nullable = false)
    private LocalDate finishDate;

    @Column(name="status", nullable = false)
    private int status;

    // Getters y setters...
}
```

Listing 4: Entidad Event

4.4 API REST

4.4.1 Endpoints Principales

Método	Endpoint	Controlador	Descripción
POST	/api/auth/login	LoginController	Autenticación de usuario
POST	/api/auth/register	RegisterController	Registro de nuevo usuario
GET	/api/events	EventController	Listado de eventos con filtros
POST	/api/events	EventController	Creación de nuevo evento
PUT	/api/events/{id}	EventController	Actualización de evento
DELETE	/api/events/{id}	EventController	Eliminación de evento
GET	/api/users/profile	UserController	Perfil del usuario actual
PUT	/api/users/profile	UserController	Actualización de perfil
POST	/api/organizer-petitions	OrganizerPetitionController	Solicitud de organizador
GET	/api/categories	CategoryController	Listado de categorías
POST	/api/tickets	TicketController	Compra de tickets
GET	/api/notifications	NotificationController	Notificaciones del usuario

4.4.2 Ejemplo de Controlador

```
@RestController
@RequestMapping("/api/events")
@CrossOrigin(origins = "*")
public class EventController {

```

```
    @Autowired
    private EventService eventService;

    @GetMapping
    public ResponseEntity<ResponseDto> getAllEvents(
            @RequestParam(required = false) String search,
            @RequestParam(required = false) Long categoryId,
            @RequestParam(required = false) String city,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {

        Page<EventDto> events = eventService.getAllEvents(search,
    categoryId, city, page, size);
        return ResponseEntity.ok(new ResponseDto("Events retrieved
    successfully", events));
    }

    @PostMapping
    public ResponseEntity<ResponseDto> createEvent(@RequestBody EventDto
    eventDto) {
        EventDto createdEvent = eventService.createEvent(eventDto);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(new ResponseDto("Event created successfully",
    createdEvent));
    }
}
```

Listing 5: EventController

4.5 Servicios

4.5.1 EventService

```
public interface EventService {
    Page<EventDto> getAllEvents(String search, Long categoryId, String city
    , int page, int size);
    EventDto getEventById(Long id);
    EventDto createEvent(EventDto eventDto);
    EventDto updateEvent(Long id, EventDto eventDto);
    void deleteEvent(Long id);
    List<EventDto> getEventsByOrganizer(Long organizerId);
}
```

Listing 6: EventService Interface

4.5.2 Implementación del Servicio

```
@Service
public class EventServiceImpl implements EventService {

    @Autowired
    private EventRepository eventRepository;

    @Autowired
    private UserRepository userRepository;

```

```
    @Override
    public Page<EventDto> getAllEvents(String search, Long categoryId,
    String city, int page, int size) {
        Pageable pageable = PageRequest.of(page, size);
        return eventRepository.findAllWithFilters(search, categoryId, city,
     pageable)
                .map(this::convertToDto);
    }

    private EventDto convertToDto(Event event) {
        EventDto dto = new EventDto();
        // Mapeo de propiedades...
        return dto;
    }
}
```

Listing 7: EventServiceImpl

5 Frontend Web - Arquitectura Detallada

5.1 Estructura del Proyecto

La estructura del proyecto frontend sigue las mejores prácticas de React:

- `public/`: Archivos estáticos (HTML, assets)
- `src/components/`: Componentes reutilizables
- `src/components/layout/`: Componentes de layout y navegación
- `src/components/views/`: Páginas principales de la aplicación
- `src/context/`: Context API para estado global
- `App.jsx`: Configuración de rutas y estructura principal
- `main.jsx`: Punto de entrada de React

5.2 Configuración Principal

5.2.1 package.json

```
{
  "name": "tuevento-frontend-web",
  "version": "0.0.1",
  "type": "module",
  "dependencies": {
    "react": "^19.1.1",
    "react-dom": "^19.1.1",
    "react-router-dom": "^7.8.1",
    "framer-motion": "^12.23.16",
    "lucide-react": "^0.460.0",
    "immer": "^10.1.1",
    "interactjs": "^1.10.27",
    "nanoid": "^5.0.8"
  },
```

```
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.3",
    "vite": "^6.0.1",
    "tailwindcss": "^4.1.12",
    "eslint": "^9.15.0"
  }
}
```

Listing 8: Dependencias del Proyecto

5.2.2 vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: true,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          router: ['react-router-dom'],
          ui: ['lucide-react', 'framer-motion']
        }
      }
    }
  },
  server: {
    port: 3000,
    host: true
  }
})
```

Listing 9: Configuración de Vite

5.3 Componentes Principales

5.3.1 App.jsx - Punto de Entrada

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import MainLayout from './components/layout/MainLayout';
import LandingPage from './components/views/LandingPage';
import Events from './components/views/Events';
import EventsInfo from './components/views/EventsInfo';
import FloorPlanDesigner from './components/views/FloorPlanDesigner';
import AdminLogin from './components/views/AdminLogin';
import AdminDashboard from './components/views/AdminDashboard';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<MainLayout><LandingPage /></MainLayout>}
   />
```

```
        <Route path="/events" element={<MainLayout><Events /></MainLayout>} />
        <Route path="/event-info" element={<MainLayout><EventsInfo /></MainLayout>} />
        <Route path="/FloorPlanDesigner" element={<FloorPlanDesigner />} />
        <Route path="/admin-login" element={<AdminLogin />} />
        <Route path="/admin-dashboard" element={<AdminDashboard />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Listing 10: Configuración de Rutas

5.3.2 Events.jsx - Listado de Eventos

```
import { useState, useEffect } from 'react';
import EventCard from './EventCard';

const Events = () => {
  const [events, setEvents] = useState([]);
  const [filteredEvents, setFilteredEvents] = useState([]);
  const [activeFilter, setActiveFilter] = useState('Ciudad');
  const [selectedCity, setSelectedCity] = useState('Bogot\'{a}');
  const [searchTerm, setSearchTerm] = useState('');

  useEffect(() => {
    // Cargar eventos desde API
    fetchEvents();
  }, []);

  const fetchEvents = async () => {
    try {
      const response = await fetch('/api/events');
      const data = await response.json();
      setEvents(data);
      setFilteredEvents(data);
    } catch (error) {
      console.error('Error fetching events:', error);
    }
  };

  const handleFilter = () => {
    let filtered = events.filter(event => {
      if (selectedCity !== 'Bogot\'{a}' && event.city !== selectedCity) return false;
      if (searchTerm && !event.title.toLowerCase().includes(searchTerm.toLowerCase())) return false;
      return true;
    });
    setFilteredEvents(filtered);
  };

  return (
    <div className="container mx-auto px-4 py-8">
```

```
      {/* Filtros y b\'{u}squeda */}
      <div className="mb-8">
        <input
          type="text"
          placeholder="Buscar eventos..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
          className="w-full p-3 border rounded-lg"
        />
        <button onClick={handleFilter} className="mt-4 bg-purple-600 text-white px-6 py-2 rounded-lg">
          Filtrar
        </button>
      </div>

      {/* Grid de eventos */}
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {filteredEvents.map(event => (
          <EventCard key={event.id} event={event} />
        ))}
      </div>
    </div>
  );
};

export default Events;
```

Listing 11: Componente Events

5.4 Context API y Estado Global

5.4.1 MapContext.jsx

```
import React, { createContext, useContext, useState } from 'react';

const MapContext = createContext();

export const MapProvider = ({ children }) => {
  const [elements, setElements] = useState([]);
  const [selectedId, setSelectedId] = useState(null);

  const dispatch = (action) => {
    switch (action.type) {
      case 'SET_ELEMENTS':
        setElements(action.payload);
        break;
      case 'SET_SELECTED':
        setSelectedId(action.payload);
        break;
      case 'ADD_ELEMENT':
        setElements(prev => [...prev, action.payload]);
        break;
      case 'UPDATE_ELEMENT':
        setElements(prev => prev.map(el =>
          el.id === action.payload.id ? { ...el, ...action.payload } : el
        ));
```

```
        break;
      case 'DELETE_ELEMENT':
        setElements(prev => prev.filter(el => el.id !== action.payload));
        break;
      default:
        break;
    }
  };

  return (
    <MapContext.Provider value={{ elements, selectedId, dispatch }}>
      {children}
    </MapContext.Provider>
  );
};

export const useMapState = () => useContext(MapContext);
export const useMapDispatch = () => {
  const { dispatch } = useContext(MapContext);
  return dispatch;
};
```

Listing 12: Context para el Diseñador de Mapas

5.5 Hooks Personalizados

5.5.1 useHistory Hook

```
import { useState } from 'react';

const useHistory = (initialState) => {
  const [history, setHistory] = useState([initialState]);
  const [currentIndex, setCurrentIndex] = useState(0);

  const pushState = (newState) => {
    const newHistory = history.slice(0, currentIndex + 1);
    newHistory.push(newState);
    setHistory(newHistory);
    setCurrentIndex(newHistory.length - 1);
  };

  const undo = () => {
    if (currentIndex > 0) {
      setCurrentIndex(currentIndex - 1);
    }
  };

  const redo = () => {
    if (currentIndex < history.length - 1) {
      setCurrentIndex(currentIndex + 1);
    }
  };

  return {
    state: history[currentIndex],
    pushState,
    undo,
```

```
    redo,
    canUndo: currentIndex > 0,
    canRedo: currentIndex < history.length - 1
  };
};

export default useHistory;
```

Listing 13: Hook para Undo/Redo

5.5.2 useDragAndDrop Hook

```
import { useState } from 'react';

const useDragAndDrop = (elements, onUpdate) => {
  const [draggedElement, setDraggedElement] = useState(null);

  const handleMouseDown = (elementId) => {
    setDraggedElement(elementId);
  };

  const handleMouseMove = (deltaX, deltaY) => {
    if (draggedElement) {
      onUpdate(draggedElement, {
        x: elements[draggedElement].x + deltaX,
        y: elements[draggedElement].y + deltaY
      });
    }
  };

  const handleMouseUp = () => {
    setDraggedElement(null);
  };

  return {
    handleMouseDown,
    handleMouseMove,
    handleMouseUp,
    isDragging: draggedElement !== null
  };
};

export default useDragAndDrop;
```

Listing 14: Hook para Drag and Drop

6 Base de Datos

6.1 Relaciones

- **User - Event**: Un usuario puede organizar múltiples eventos (1:N)
- **Event - Location**: Un evento ocurre en una ubicación (N:1)
- **Event - EventImg**: Un evento puede tener múltiples imágenes (1:N)

- **Event - CategoryEvent**: Un evento puede tener múltiples categorías (1:N)
- **Event - EventRating**: Un evento puede tener múltiples calificaciones (1:N)
- **User - Ticket**: Un usuario puede comprar múltiples tickets (1:N)
- **Event - Ticket**: Un evento puede tener múltiples tickets vendidos (1:N)

6.2 Datos Iniciales

El sistema incluye datos iniciales para desarrollo y testing:

- Usuario administrador por defecto
- Categorías de eventos predefinidas
- Datos geoespaciales (departamentos, ciudades)
- Direcciones y ubicaciones de ejemplo

7 Instalación

7.1 Prerrequisitos

7.1.1 Backend

- Java 17 JDK
- Maven 3.6+
- PostgreSQL 12+
- Cuenta AWS con S3
- Cuenta Gmail para SMTP

7.1.2 Frontend Web

- Node.js 18+
- npm o yarn

7.1.3 Frontend Móvil

- Node.js 18+
- Expo CLI
- Android Studio (para Android)
- Xcode (para iOS)

7.2 Instalación del Backend

7.2.1 1. Clonar el repositorio

```
git clone https://github.com/lozano2303/tu-evento.git
cd tuevento-backend
```

7.2.2 2. Configurar base de datos

```
CREATE DATABASE tuevento;
CREATE USER tuevento_user WITH PASSWORD 'password';
GRANT ALL PRIVILEGES ON DATABASE tuevento TO tuevento_user;
```

7.2.3 3. Configurar variables de entorno

```
# Crear archivo .env en la raíz del proyecto
DATABASE_URL=jdbc:postgresql://localhost:5432/tuevento
DATABASE_USERNAME=tuevento_user
DATABASE_PASSWORD=password
JWT_SECRET=your-secret-key
AWS_ACCESS_KEY=your-aws-key
AWS_SECRET_KEY=your-aws-secret
EMAIL_USERNAME=your-email@gmail.com
EMAIL_PASSWORD=your-app-password
```

7.2.4 4. Compilar y ejecutar

```
mvn clean install
mvn spring-boot:run
```

7.3 Instalación del Frontend Web

7.3.1 1. Clonar el repositorio

```
git clone https://github.com/tu-organizacion/tuevento-frontend-web.git
cd tuevento-frontend-web
```

7.3.2 2. Instalar dependencias

```
npm install
```

7.3.3 3. Configurar variables de entorno

```
# Crear archivo .env.local
VITE_API_BASE_URL=http://localhost:8080/api
```

7.3.4 4. Ejecutar en modo desarrollo

```
npm run dev
```

7.3.5 5. Construir para producción

```
npm run build
```

8 Seguridad

8.1 Autenticación y Autorización

8.1.1 JWT Authentication

El sistema utiliza JSON Web Tokens para autenticación stateless:

- **Login**: Genera token con claims de usuario y expiración
- **Verificación**: Valida token en cada request protegido
- **Refresh**: Extiende sesión sin re-autenticación completa

8.1.2 Roles y Permisos

USER Usuario regular - acceso básico a eventos y perfil

ORGANIZER Organizador - gestión de eventos propios

ADMIN Administrador - acceso completo al sistema

8.2 Validación de Datos

8.2.1 Backend Validation

- Validación a nivel de controlador con Bean Validation
- Validación de negocio en servicios
- Sanitización de inputs para prevenir XSS

8.2.2 Frontend Validation

- Validación en tiempo real en formularios
- Mensajes de error descriptivos
- Prevención de envío de datos inválidos

8.3 Seguridad de Infraestructura

8.3.1 CORS Configuration

```java
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:3000", "https://tuevento.com")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

Listing 15: Configuración CORS

8.3.2 Content Security Policy

```javascript
const csp = {
  directives: {
    defaultSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"],
    fontSrc: ["'self'", "https://fonts.gstatic.com"],
    imgSrc: ["'self'", "data:", "https://*.amazonaws.com"],
    scriptSrc: ["'self'"],
    connectSrc: ["'self'", "https://api.tuevento.com"]
  }
};
```

Listing 16: CSP Headers

9 Conclusión

9.1 Logros del Sistema

TuEvento representa una implementación completa y moderna de un sistema de gestión de eventos que incluye:

- **Arquitectura escalable**: Separación clara entre frontend y backend
- **Tecnologías actuales**: Uso de frameworks modernos y mejores prácticas
- **Seguridad robusta**: Autenticación JWT y autorización basada en roles
- **Experiencia de usuario**: Interfaces responsivas y intuitivas
- **Funcionalidades completas**: Desde registro hasta gestión de eventos

9.2 Futuras Mejoras

9.2.1 Corto Plazo

- Sistema de pagos integrado
- Notificaciones push

- API de terceros para mapas
- Sistema de reseñas avanzado

—

Fecha de creación: 3 de octubre de 2025
Versión del documento: 1.0
Autor: Equipo de Desarrollo TuEvento